

Laporan Analisis dan Refactoring Kode

Aplikasi Portal Kuesioner Psikometrik PointMarket

Jenis Dokumen	Contoh laporan praktikum/proyek aplikasi web
Topik	MVC, SOLID, Clean Code, High Cohesion, Low Coupling
Sumber Observasi	Portal Kuesioner PM - PHP native MVC
Tanggal	21 Juni 2026

Catatan: dokumen ini adalah artefak pembelajaran. Contoh refactoring ditulis sebagai rancangan edukatif dan tidak mengubah kode aplikasi aslinya.

Dokumen ini disusun sebagai contoh laporan praktikum/proyek untuk topik analisis dan refactoring kode aplikasi web. Studi kasus yang digunakan adalah Portal Kuesioner PM, yaitu portal kuesioner psikometrik yang menjadi bagian dari ekosistem riset PointMarket.

Catatan penting: contoh refactoring pada dokumen ini bersifat edukatif. Kode aplikasi riset tidak diubah. Potongan "sesudah refactoring" menunjukkan rancangan perbaikan yang dapat dikerjakan pada branch latihan terpisah oleh mahasiswa.

1. Identitas Proyek

Komponen	Isi
Nama Aplikasi	Portal Kuesioner Psikometrik PointMarket
Jenis Aplikasi	Aplikasi Web
Pola Arsitektur	PHP native MVC
Topik Praktikum	MVC, SOLID, Clean Code, High Cohesion, Low Coupling
Nama Kelompok	<i>Diisi oleh mahasiswa</i>
Anggota Kelompok	<i>Diisi oleh mahasiswa</i>
Repository	<i>Diisi sesuai repository kelompok atau repository latihan</i>
Sumber Observasi Kode	<i>Diisi oleh mahasiswa</i>
Tanggal Revisi	<i>Diisi oleh mahasiswa</i>

2. Deskripsi Singkat Aplikasi

Portal Kuesioner Psikometrik PointMarket merupakan aplikasi web berbasis PHP dan MySQL yang digunakan untuk mengelola pengisian serta pelaporan hasil kuesioner psikometrik mahasiswa. Berdasarkan struktur repository, aplikasi ini memuat modul VARK, MSLQ, dan AMS. Ketiga instrumen

tersebut digunakan untuk mencatat profil gaya belajar, skor strategi/motivasi belajar, dan tipe motivasi akademik mahasiswa.

Fitur yang teridentifikasi dari repository meliputi:

No	Fitur	Bukti File/Modul
1	Login dan session mahasiswa	app/Controllers/HomeController.php, app/Models/Student.php
2	Login dan dashboard admin	app/Controllers/AdminController.php, app/Views/admin/index.php
3	Kuesioner VARK	app/Controllers/VarkController.php, app/Views/vark/index.php, vark_questions
4	Kuesioner MSLQ	app/Controllers/MslqController.php, app/Views/mslq/index.php, mslq_questions
5	Kuesioner AMS	app/Controllers/AmsController.php, app/Views/ams/index.php, ams_questions
6	Penyimpanan respons	app/Models/Question.php, tabel responses
7	Perhitungan dan penyimpanan hasil	app/Models/Student.php, app/Controllers/ResultsController.php
8	API hasil kuesioner	app/Controllers/ApiController.php, dev-resources/docs/swagger.yaml
9	Pengaturan buka/tutup instrumen	app/Models/Setting.php, app/Controllers/AdminController.php
10	Dashboard ringkasan admin	AdminController::index()

Aplikasi berjalan melalui Docker. File `docker-compose.yml` menunjukkan layanan aplikasi pada port 8085, database MySQL pada port host 3308, dan phpMyAdmin pada port 8086.

3. Tujuan Refactoring

Refactoring pada studi kasus ini bertujuan untuk:

- 1) Memperjelas tanggung jawab antara controller, model, service, repository, dan validator.
- 2) Mengurangi controller yang terlalu banyak menangani proses sekaligus.
- 3) Memisahkan logika submit kuesioner, skoring psikometrik, histori, dan format output.
- 4) Mengurangi duplikasi alur submit pada VARK, MSLQ, dan AMS.
- 5) Menyatukan validasi jawaban kuesioner agar aturan input lebih konsisten.
- 6) Memindahkan query laporan yang kompleks dari controller ke repository/service.
- 7) Membuat kode lebih mudah diuji, dibaca, dan dikembangkan untuk instrumen psikometrik baru.

4. Ruang Lingkup Analisis Kode

Analisis difokuskan pada lima area yang paling terlihat dari repository.

No	Modul	File/Method	Alasan Dipilih
1	Pengisian VARK	VarkController::submit()	Controller menangani simpan jawaban, proses VARK NLP fusion, pencatatan histori, session, dan redirect
2	Pengisian MSLQ dan AMS	MslqController::submit(), AmsController::submit()	Alur submit mirip dan berulang pada beberapa controller
3	Laporan Hasil	ResultsController::index()	Controller menghitung hasil, melakukan query, memperbarui skor, dan membentuk JSON
4	Dashboard Admin	AdminController::index()	Query statistik, agregasi, transformasi label, dan pagination bercampur dalam controller
5	Manajemen Butir Pertanyaan	AdminController::vark(), AdminController::mslq(), AdminController::ams(), delete_question()	CRUD pertanyaan berulang dan akses tabel tersebar di controller

5. Struktur Folder Aplikasi

Struktur aktual yang teridentifikasi:

```

kuisisioner_pm/
|-- app/
|   |-- Controllers/
|   |   |-- AdminController.php
|   |   |-- AmsController.php
|   |   |-- ApiController.php
|   |   |-- DashboardController.php
|   |   |-- HomeController.php
|   |   |-- MslqController.php
|   |   |-- ResultsController.php
|   |   |-- VarkController.php
|   |-- Core/
|   |   |-- Controller.php
|   |   |-- Database.php
|   |   |-- Model.php
|   |   |-- Router.php
|   |-- Helpers/
|   |   |-- VarkNlpHelper.php
|   |-- Models/
|   |   |-- ClassModel.php
|   |   |-- Question.php
|   |   |-- Setting.php
|   |   |-- Student.php
|   |-- Views/
|   |   |-- admin/
|   |   |-- ams/
|   |   |-- api/
|   |   |-- dashboard/
|   |   |-- errors/
|   |   |-- home/
|   |   |-- layout/
|   |   |-- mslq/
|   |   |-- results/
|   |   |-- vark/
|-- database/
|   |-- db_schema.sql
|   |-- seed_data.sql
|-- dev-resources/
|   |-- docs/
|   |-- swagger.yaml

```

```
| |-- legacy/
|-- Docs/
|-- public/
| |-- index.php
| |-- css/
|-- scripts/
|-- docker-compose.yml
|-- Dockerfile
|-- README.md
```

Catatan arsitektur: repository belum memperlihatkan folder `Services`, `Repositories`, atau `Validators`. Oleh karena itu, bagian refactoring mengusulkan penambahan lapisan tersebut sebagai arah perbaikan modular, bukan sebagai kondisi yang sudah ada.

6. Ringkasan Arsitektur MVC

Aplikasi menggunakan pola MVC sederhana dengan entry point di `public/index.php`, autoloader PSR-4 sederhana, dan router pada `app/Core/Router.php`.

Lapisan	Contoh File	Tanggung Jawab Saat Ini
Entry Point	<code>public/index.php</code>	Memulai session, mendaftarkan autoloader, dan membuat router
Router	<code>app/Core/Router.php</code>	Memetakan URL ke controller dan method
Controller	<code>VarkController</code> , <code>MslqController</code> , <code>AmsController</code> , <code>AdminController</code>	Menerima request, memanggil model, menyiapkan data view, melakukan redirect
Model	<code>Student</code> , <code>Question</code> , <code>Setting</code> , <code>ClassModel</code>	Mengakses database dan sebagian logika domain seperti skoring
Helper	<code>VarkNlpHelper</code>	Membantu fusion hasil VARK dengan narasi NLP
View	<code>app/Views/...</code>	Menampilkan form kuesioner, dashboard, halaman admin, dan hasil
Database	<code>database/db_schema.sql</code>	Mendefinisikan tabel <code>students</code> , <code>admins</code> , <code>system_settings</code> , <code>vark_questions</code> , <code>mslq_questions</code> , <code>ams_questions</code> , dan <code>responses</code>

Ringkasan alur utama:

- 1) Mahasiswa login melalui `HomeController`.
- 2) Mahasiswa memilih instrumen, misalnya VARK, MSLQ, atau AMS.
- 3) Controller instrumen mengambil daftar pertanyaan dari `Question`.
- 4) Jawaban disimpan ke tabel `responses`.
- 5) Hasil dihitung dan diperbarui ke tabel `students`.
- 6) Dashboard dan API mengambil ringkasan hasil dari database.

7. Daftar Temuan Masalah Kode

No	File/Method	Masalah Kode	Prinsip Terkait	Dampak Negatif
1	VarkController::submit()	Submit VARK melakukan terlalu banyak proses dalam controller	SRP, Separation of Concerns, High Cohesion	Sulit diuji dan sulit dipelihara ketika aturan VARK berubah
2	MslqController::submit() dan AmsController::submit()	Alur submit, simpan jawaban, histori, session, dan redirect berulang	DRY, Clean Code	Perubahan format submit harus dilakukan di banyak controller
3	ResultsController::index()	Query hasil, perhitungan fallback, update skor, dan JSON disatukan	SRP, Low Coupling	Controller bergantung pada detail query dan aturan hasil
4	AdminController::index()	Query dashboard sangat banyak dan transformasi data dilakukan di controller	SRP, Low Coupling	Dashboard sulit diuji dan sulit dikembangkan
5	AdminController::vark()/mslq()/ams()	CRUD pertanyaan per instrumen berulang, validasi input belum terpusat	DRY, OCP, Clean Code	Menambah instrumen baru berpotensi menambah method serupa

8. Analisis Before-After Refactoring

8.1 Temuan 1 - Controller Terlalu Gemuk pada Submit VARK

Lokasi Kode

app/Controllers/VarkController.php, method submit().

Kode Sebelum Refactoring

Blok kode (php)

```
public function submit()
{
    if ($_SERVER['REQUEST_METHOD'] == 'POST') {
        $studentModel = new \App\Models\Student();
        $questionModel = new Question();
        $answers = $_POST['answers'] ?? [];
        $narrative = $_POST['vark_narrative'] ?? '';

        foreach ($answers as $q_id => $val) {
            $questionModel->saveResponse($_SESSION['student_id'], 'VARK', $q_id, $val);
        }

        $result = $studentModel->processVarkNlpFusion($_SESSION['student_id'], $narrative);

        $db = \App\Core\Database::getInstance();
        $stmt = $db->prepare("INSERT INTO quiz_history (student_id, quiz_type, result_label) VALUES
        (?, 'VARK', ?)");
        $stmt->execute([$_SESSION['student_id'], $result]);

        $_SESSION['quiz_success'] = 'VARK Assessment';
        $this->redirect('/dashboard/profile');
    }
}
```

Masalah yang Ditemukan

Method ini tidak hanya menerima request, tetapi juga membaca input mentah, menyimpan jawaban, memanggil proses fusion VARK NLP, menulis histori, mengatur session, dan menentukan redirect. Hal ini membuat controller memiliki terlalu banyak alasan untuk berubah.

Prinsip yang Dilanggar

- 1) Single Responsibility Principle: controller menangani logika request sekaligus proses domain.
- 2) Separation of Concerns: penyimpanan, skoring, histori, dan response bercampur.
- 3) High Cohesion: tanggung jawab method tidak fokus pada satu jenis pekerjaan.

Strategi Refactoring

- 1) Buat `QuestionnaireResponseValidator` untuk validasi input.
- 2) Buat `QuestionnaireSubmissionService` untuk alur submit umum.
- 3) Buat `PsychometricScoringService` untuk pemilihan strategi skoring.
- 4) Buat `QuizHistoryRepository` untuk pencatatan histori.
- 5) Controller hanya menerima request dan mengarahkan response.

Kode Sesudah Refactoring

Blok kode (php)

```
public function submit()
{
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
        $this->redirect('/vark');
    }

    $result = $this->submissionService->submitVark(
        (int) $_SESSION['student_id'],
        $_POST['answers'] ?? [],
        $_POST['vark_narrative'] ?? ''
    );

    $_SESSION['quiz_success'] = 'VARK Assessment';
    $this->redirect('/dashboard/profile');
}
```

Blok kode (php)

```
class QuestionnaireSubmissionService
{
    public function __construct(
        private QuestionnaireResponseValidator $validator,
        private ResponseRepository $responseRepository,
        private PsychometricScoringService $scoringService,
        private QuizHistoryRepository $historyRepository
    ) {}

    public function submitVark(int $studentId, array $answers, string $narrative): string
    {
        $validAnswers = $this->validator->validateVarkAnswers($answers);

        $this->responseRepository->replaceResponses($studentId, 'VARK', $validAnswers);

        $resultLabel = $this->scoringService->calculateVarkResult($studentId, $narrative);

        $this->historyRepository->recordLabel($studentId, 'VARK', $resultLabel);

        return $resultLabel;
    }
}
```

```
}

```

Dampak Perbaikan

Controller menjadi lebih ringkas dan fokus. Perubahan aturan VARK, misalnya perubahan threshold NLP atau cara penyimpanan histori, tidak lagi memerlukan perubahan langsung pada controller.

8.2 Temuan 2 - Duplikasi Alur Submit pada MSLQ dan AMS

Lokasi Kode

app/Controllers/MslqController.php::submit() dan
app/Controllers/AmsController.php::submit().

Kode Sebelum Refactoring

Blok kode (php)

```
public function submit()
{
    if ($_SERVER['REQUEST_METHOD'] == 'POST') {
        $studentModel = new \App\Models\Student();
        $questionModel = new Question();
        foreach ($_POST['answers'] ?? [] as $q_id => $val) {
            $questionModel->saveResponse($_SESSION['student_id'], 'MSLQ', $q_id, $val);
        }
        $score = $studentModel->calculateAndSetMslq($_SESSION['student_id']);

        $db = \App\Core\Database::getInstance();
        $stmt = $db->prepare("INSERT INTO quiz_history (student_id, quiz_type, result_label,
result_value) VALUES (?, 'MSLQ', ?, ?)");
        $stmt->execute([$_SESSION['student_id'], $score, $score]);

        $_SESSION['quiz_success'] = 'MSLQ Evaluation';
        $this->redirect('/dashboard/profile');
    }
}
```

Blok kode (php)

```
public function submit()
{
    if ($_SERVER['REQUEST_METHOD'] == 'POST') {
        $studentModel = new \App\Models\Student();
        $questionModel = new Question();
        foreach ($_POST['answers'] ?? [] as $q_id => $val) {
            $questionModel->saveResponse($_SESSION['student_id'], 'AMS', $q_id, $val);
        }
        $result = $studentModel->calculateAndSetAms($_SESSION['student_id']);

        $db = \App\Core\Database::getInstance();
        $stmt = $db->prepare("INSERT INTO quiz_history (student_id, quiz_type, result_label) VALUES
(?, 'AMS', ?)");
        $stmt->execute([$_SESSION['student_id'], $result]);

        $_SESSION['quiz_success'] = 'AMS Motivation';
        $this->redirect('/dashboard/profile');
    }
}
```

Masalah yang Ditemukan

MSLQ dan AMS memiliki pola submit yang hampir sama: membaca `answers`, menyimpan respons, menghitung hasil, menulis `quiz_history`, mengatur session, dan redirect. Perbedaannya hanya pada tipe instrumen, metode skoring, dan format hasil.

Selain itu, variabel seperti `$q_id` dan `$val` masih dapat dibuat lebih bermakna dalam contoh pembelajaran, misalnya `$questionId` dan `$answerValue`.

Prinsip yang Dilanggar

- 1) DRY: pola submit berulang.
- 2) Clean Code: validasi jawaban belum terpusat dan nama variabel input masih terlalu singkat untuk pembelajaran.
- 3) OCP: menambah instrumen baru berpotensi menambah controller submit baru dengan pola serupa.

Strategi Refactoring

- 1) Buat satu service submit berbasis tipe instrumen.
- 2) Gunakan enum/constant untuk tipe instrumen: `VARK`, `MSLQ`, `AMS`.
- 3) Validasi jawaban dipusatkan berdasarkan skala instrumen.
- 4) Histori hasil ditulis melalui repository.

Kode Sesudah Refactoring

Blok kode (php)

```
public function submit()
{
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
        $this->redirect('/mslq');
    }

    $score = $this->submissionService->submit(
        studentId: (int) $_SESSION['student_id'],
        instrumentType: InstrumentType::MSLQ,
        answers: $_POST['answers'] ?? []
    );

    $_SESSION['quiz_success'] = 'MSLQ Evaluation';
    $this->redirect('/dashboard/profile');
}
```

Blok kode (php)

```
class QuestionnaireSubmissionService
{
    public function submit(int $studentId, string $instrumentType, array $answers):
        QuestionnaireResult
    {
        $validAnswers = $this->validator->validateByInstrument($instrumentType, $answers);

        $this->responseRepository->replaceResponses($studentId, $instrumentType, $validAnswers);

        $result = $this->scoringService->calculate($studentId, $instrumentType);

        $this->historyRepository->record($studentId, $instrumentType, $result);

        return $result;
    }
}
```


}

Blok kode (php)

```

final class QuestionnaireResponseValidator
{
    public function validateByInstrument(string $instrumentType, array $answers): array
    {
        if ($answers === []) {
            throw new InvalidArgumentException('Jawaban kuesioner tidak boleh kosong.');
```

Dampak Perbaikan

Alur submit menjadi konsisten. Jika ada instrumen baru, mahasiswa cukup menambahkan konfigurasi dan strategi skoring baru tanpa menggandakan struktur controller.

8.3 Temuan 3 - Logika Hasil dan Query Bercampur di ResultsController**Lokasi Kode**

app/Controllers/ResultsController.php::index().

Kode Sebelum Refactoring**Blok kode (php)**

```

public function index()
{
    if (!isset($_SESSION['student_id'])) {
        $this->redirect('/home/');
    }
    $db = Database::getInstance();
    $student_id = $_SESSION['student_id'];

    $current = $db->prepare("SELECT vark_type FROM students WHERE id = ?");
    $current->execute([$student_id]);
    $vark_type = $current->fetchColumn();

    if (!$vark_type) {
        $vark = $db->prepare("SELECT nilai_jawaban FROM responses WHERE student_id=? AND
tipe_pertanyaan='VARK' GROUP BY nilai_jawaban ORDER BY COUNT(*) DESC LIMIT 1");
        $vark->execute([$student_id]);
        $vark_type = $vark->fetchColumn() ?: 'Unknown';
    }

    $mslq = $db->prepare("SELECT AVG(CAST(nilai_jawaban AS UNSIGNED)) FROM responses WHERE
student_id=? AND tipe_pertanyaan='MSLQ'");
    $mslq->execute([$student_id]);
    $mslq_score = round($mslq->fetchColumn() ?: 0, 2);

    $ams = $db->prepare("SELECT q.kategori FROM responses r JOIN ams_questions q ON
r.question_id=q.id WHERE r.student_id=? AND r.tipe_pertanyaan='AMS' GROUP BY q.kategori ORDER BY
AVG(CAST(r.nilai_jawaban AS UNSIGNED)) DESC LIMIT 1");
    $ams->execute([$student_id]);
    $ams_type = $ams->fetchColumn() ?: 'Unknown';
}
```

```
(new Student())->updateScores($student_id, $vark_type, $mslq_score, $ams_type);

$this->view('results/index', [
    'vark' => $vark_type,
    'mslq' => $mslq_score,
    'ams' => $ams_type,
    'json' => json_encode(['npm' => $_SESSION['npm'], 'vark' => $vark_type, 'mslq' =>
    $mslq_score, 'ams' => $ams_type], JSON_PRETTY_PRINT)
]);
}
```

Masalah yang Ditemukan

Method `index()` menanggung banyak tugas: mengecek session, menjalankan query VARK, menghitung MSLQ, menentukan AMS, memperbarui tabel `students`, dan membentuk data JSON untuk view. Controller menjadi sangat bergantung pada struktur tabel `responses`, `students`, dan `ams_questions`.

Prinsip yang Dilanggar

- SRP: controller tidak hanya mengatur halaman hasil.
- Low Coupling: controller terlalu dekat dengan detail database.
- Clean Code: query panjang menurunkan keterbacaan method.

Strategi Refactoring

- Pindahkan query hasil ke `QuestionnaireResultRepository`.
- Pindahkan aturan fallback dan agregasi ke `PsychometricResultService`.
- Bentuk payload JSON melalui presenter/formatter, misalnya `QuestionnaireResultPresenter`.

Kode Sesudah Refactoring

Blok kode (php)

```
public function index()
{
    if (!isset($_SESSION['student_id'])) {
        $this->redirect('/home');
    }

    $result = $this->resultService->getCurrentResult((int) $_SESSION['student_id']);

    $this->view('results/index', [
        'vark' => $result->varkType,
        'mslq' => $result->mslqScore,
        'ams' => $result->amsType,
        'json' => $this->resultPresenter->toJson($result, $_SESSION['npm'])
    ]);
}
```

Blok kode (php)

```
class PsychometricResultService
{
    public function __construct(
        private QuestionnaireResultRepository $resultRepository,
        private StudentRepository $studentRepository
    ) {}

    public function getCurrentResult(int $studentId): PsychometricResult
    {
        $result = new PsychometricResult(
            varkType: $this->resultRepository->findVarkResult($studentId),
```

```

        mslqScore: $this->resultRepository->calculateMslqAverage($studentId),
        amsType: $this->resultRepository->findDominantAmsType($studentId)
    );

    $this->studentRepository->updatePsychometricSummary($studentId, $result);

    return $result;
}
}

```

Dampak Perbaikan

Controller hanya menampilkan hasil. Detail query dan aturan perhitungan dapat diuji secara terpisah pada service/repository.

8.4 Temuan 4 - Query Dashboard Admin Terlalu Banyak di Controller

Lokasi Kode

app/Controllers/AdminController.php::index().

Kode Sebelum Refactoring

Blok kode (php)

```

public function index()
{
    $studentModel = new Student();
    $db = Database::getInstance();

    $approvedCount = $studentModel->countApproved();
    $pendingCount = $studentModel->countPending();

    $data = [
        'total' => $approvedCount + $pendingCount,
        'pending' => $pendingCount,
        'settings' => (new Setting())->getAllRaw()
    ];

    $varRows = $db->query("SELECT vark_type as label, COUNT(*) as value FROM students WHERE
    vark_type IS NOT NULL AND vark_type != '' GROUP BY vark_type")->fetchAll();
    $data['ams_dist'] = $db->query("SELECT ams_type as label, COUNT(*) as value FROM students WHERE
    ams_type IS NOT NULL AND ams_type != '' GROUP BY ams_type")->fetchAll();

    $data['mslq_avg'] = $db->query("
    SELECT DATE_FORMAT(submitted_at, '%b %Y') as label, AVG(result_value) as value
    FROM quiz_history
    WHERE quiz_type = 'MSLQ'
    GROUP BY DATE_FORMAT(submitted_at, '%b %Y'), DATE_FORMAT(submitted_at, '%Y-%m')
    ORDER BY MIN(submitted_at) ASC
    LIMIT 6
    ")->fetchAll();

    $data['recent_history'] = $db->query("
    SELECT h.*, s.nama, s.npm, c.class_name
    FROM quiz_history h
    JOIN students s ON h.student_id = s.id
    LEFT JOIN classes c ON s.kelas = c.class_name
    ORDER BY h.submitted_at DESC
    LIMIT $perPage OFFSET $offset
    ")->fetchAll();

    $this->view('admin/index', $data);
}

```

Masalah yang Ditemukan

Dashboard admin membutuhkan banyak data agregat: distribusi VARK, distribusi AMS, rata-rata MSLQ, aktivitas, kelengkapan kuesioner, dan histori terbaru. Saat semua query dan transformasi label diletakkan di controller, controller menjadi pusat laporan sekaligus pengendali view.

Prinsip yang Dilanggar

- 1) SRP: controller juga menjadi report builder.
- 2) Low Coupling: controller bergantung langsung pada detail SQL.
- 3) Clean Code: method panjang dan sulit dibaca sebagai satu alur bisnis.

Strategi Refactoring

- 1) Buat `AdminDashboardService` sebagai koordinator data dashboard.
- 2) Buat `AdminDashboardRepository` untuk query statistik.
- 3) Buat `VarkLabelMapper` atau formatter untuk pemetaan label V/A/R/K.
- 4) Validasi `page`, `perPage`, dan `offset` sebelum dipakai.

Kode Sesudah Refactoring

Blok kode (php)

```
public function index()
{
    $page = max(1, (int) ($_GET['page'] ?? 1));

    $dashboard = $this->dashboardService->buildDashboard($page, 10);

    $this->view('admin/index', $dashboard->toArray());
}
```

Blok kode (php)

```
class AdminDashboardService
{
    public function __construct(
        private AdminDashboardRepository $repository,
        private VarkLabelMapper $varkLabelMapper
    ) {}

    public function buildDashboard(int $page, int $perPage): AdminDashboardData
    {
        $completion = $this->repository->getQuestionnaireCompletion();

        return new AdminDashboardData(
            totalStudents: $this->repository->countAllStudents(),
            pendingStudents: $this->repository->countPendingStudents(),
            varkDistribution: $this->varkLabelMapper->mapRows($this->repository-
>getVarkDistribution()),
            amsDistribution: $this->repository->getAmsDistribution(),
            mslqTrend: $this->repository->getMslqTrend(6),
            activity: $this->repository->getRecentActivity(30),
            completion: $completion,
            recentHistory: $this->repository->getRecentHistory($page, $perPage)
        );
    }
}
```

Dampak Perbaikan

Dashboard menjadi lebih mudah diuji. Query statistik dapat diubah tanpa menyentuh controller atau view selama format data service tetap sama.

8.5 Temuan 5 - CRUD Pertanyaan Berulang dan Validasi Belum Terpusat

Lokasi Kode

app/Controllers/AdminController.php::vark(), mslq(), ams(), dan delete_question().

Kode Sebelum Refactoring

Blok kode (php)

```
public function vark()
{
    $db = Database::getInstance();
    if ($SERVER['REQUEST_METHOD'] == 'POST') {
        $id = $_POST['id'] ?? '';
        $teks = $_POST['teks_pertanyaan'] ?? '';
        if ($id) {
            $stmt = $db->prepare("UPDATE vark_questions SET teks_pertanyaan=?, opt_v=?, opt_a=?,
            opt_r=?, opt_k=? WHERE id=?");
            $stmt->execute([$teks, $_POST['opt_v'], $_POST['opt_a'], $_POST['opt_r'],
            $_POST['opt_k'], $id]);
        } else {
            $stmt = $db->prepare("INSERT INTO vark_questions (teks_pertanyaan, opt_v, opt_a, opt_r,
            opt_k) VALUES (?, ?, ?, ?, ?)");
            $stmt->execute([$teks, $_POST['opt_v'], $_POST['opt_a'], $_POST['opt_r'],
            $_POST['opt_k']]);
        }
        $_SESSION['success'] = "Data soal VARK berhasil disimpan!";
        $this->redirect('/admin/vark');
    }
    $questions = $db->query("SELECT * FROM vark_questions")->fetchAll();
    $this->view('admin/questions_vark', ['questions' => $questions, 'title' => 'Manajemen Soal
    VARK']);
}
```

Blok kode (php)

```
public function delete_question($type, $id)
{
    $db = Database::getInstance();
    $table = $type . '_questions';
    if (in_array($type, ['vark', 'mslq', 'ams'])) {
        $stmt = $db->prepare("DELETE FROM $table WHERE id = ?");
        $stmt->execute([$id]);
        $_SESSION['success'] = "Soal berhasil dihapus!";
    }
    $this->redirect('/admin/' . $type);
}
```

Masalah yang Ditemukan

CRUD pertanyaan VARK, MSLQ, dan AMS memiliki pola serupa, tetapi ditulis sebagai method terpisah dengan query langsung di controller. Ada whitelist pada delete_question(), tetapi pemilihan tabel tetap dibentuk dari string. Validasi field belum dipusatkan, misalnya validasi teks_pertanyaan, opsi VARK, dimensi MSLQ, atau kategori AMS.

Prinsip yang Dilanggar

- 1) DRY: pola CRUD berulang.
- 2) OCP: menambah instrumen baru memerlukan penambahan method controller baru.
- 3) Clean Code: validasi input dan query berada di tempat yang sama dengan flow controller.

Strategi Refactoring

- 1) Buat konfigurasi metadata instrumen.
- 2) Buat `QuestionRepository` yang mengetahui tabel dan field per instrumen.
- 3) Buat `QuestionAdminService` untuk proses create/update/delete.
- 4) Buat `QuestionRequestValidator` untuk validasi field.

Kode Sesudah Refactoring

Blok kode (php)

```
public function questions(string $instrumentType)
{
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $this->questionAdminService->save($instrumentType, $_POST);
        $_SESSION['success'] = 'Data soal berhasil disimpan.';
        $this->redirect('/admin/' . strtolower($instrumentType));
    }

    $questions = $this->questionAdminService->list($instrumentType);

    $this->view($this->instrumentViewResolver->questionsView($instrumentType), [
        'questions' => $questions,
        'title' => $this->instrumentViewResolver->title($instrumentType)
    ]);
}
```

Blok kode (php)

```
class QuestionRepository
{
    private const TABLES = [
        'VARK' => 'vark_questions',
        'MSLQ' => 'mslq_questions',
        'AMS' => 'ams_questions',
    ];

    public function findAll(string $instrumentType): array
    {
        $table = $this->tableFor($instrumentType);
        return $this->db->query("SELECT * FROM {$table}")->fetchAll();
    }

    public function delete(string $instrumentType, int $id): bool
    {
        $table = $this->tableFor($instrumentType);
        $stmt = $this->db->prepare("DELETE FROM {$table} WHERE id = ?");
        return $stmt->execute([$id]);
    }

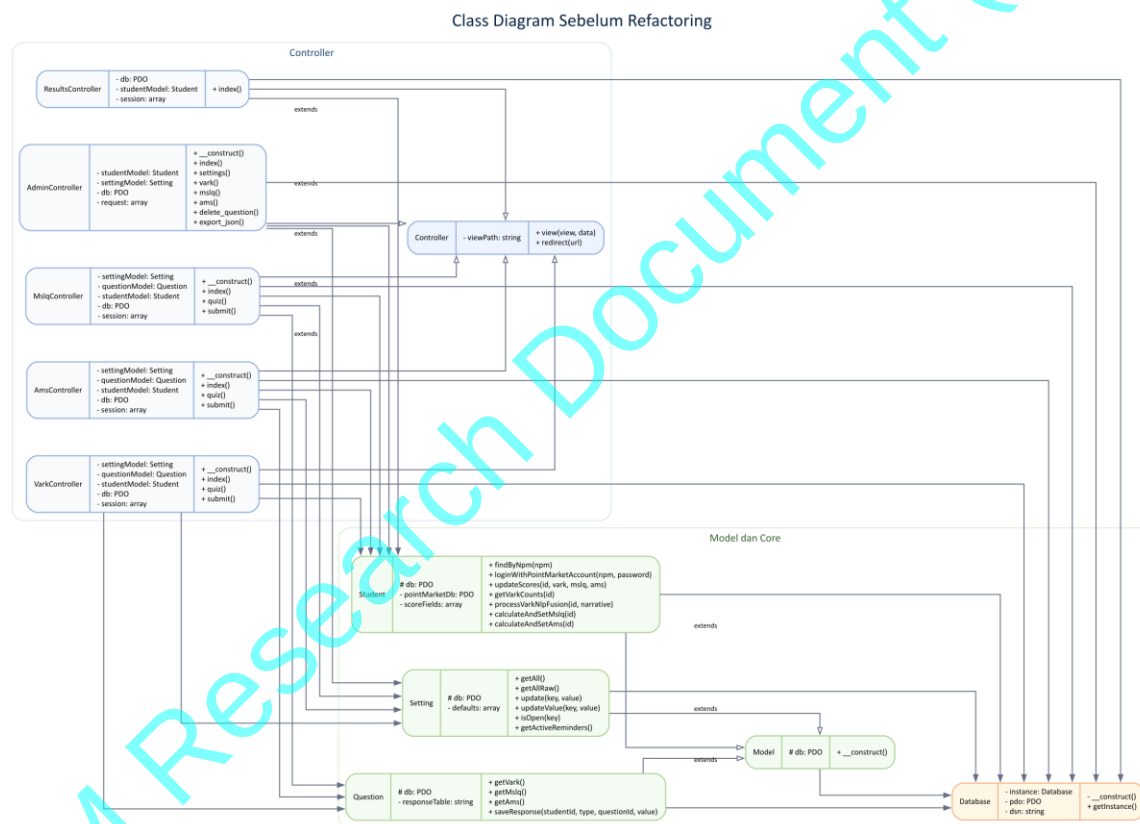
    private function tableFor(string $instrumentType): string
    {
        if (!isset(self::TABLES[$instrumentType])) {
            throw new InvalidArgumentException('Tipe instrumen tidak dikenal.');
```

Dampak Perbaikan

Controller admin lebih kecil. Tabel dan field per instrumen dikelola di satu tempat yang eksplisit. Validasi input lebih mudah dikontrol dan instrumen baru dapat ditambahkan dengan perubahan yang lebih terarah.

9. Class Diagram Sebelum Refactoring

Diagram berikut menggambarkan kondisi ringkas sebelum refactoring. Setiap class ditulis dalam format UML tiga kompartemen, yaitu nama kelas, atribut atau dependency yang digunakan kelas, dan fungsi/metode utama. Gambar dibuat dari kode Graphviz DOT agar dapat direplikasi sebagai artefak laporan teknis. Kode lengkap diagram tersedia pada docs/assets/class_diagram_sebelum_refactoring.dot.



Gambar 1. Class Diagram Sebelum Refactoring

Interpretasi: controller masih sering menjadi pusat koordinasi request, query, skoring, histori, dan format output. Atribut pada diagram sebelum refactoring dipakai untuk memperlihatkan dependency dan state yang digunakan class; sebagian dependency pada kode aktual masih dibuat sebagai objek lokal di dalam method, sehingga coupling terhadap model dan database tetap terlihat tinggi.

Prinsip SOLID	Kondisi Sebelum	Perbaikan yang Disarankan	Dampak
ISP	Belum terlihat interface khusus karena aplikasi masih sederhana	Pisahkan interface repository/service sesuai kebutuhan modul	Class tidak dipaksa bergantung pada method yang tidak dipakai
DIP	Controller bergantung langsung pada Database, Question, dan Student	Controller bergantung pada service; service bergantung pada interface repository	Dependency terhadap detail teknis berkurang

Catatan: analisis SOLID tidak perlu dipaksakan pada semua bagian kode. Pada aplikasi PHP native sederhana, SRP, OCP, dan DIP adalah area yang paling terlihat untuk latihan refactoring.

12. Analisis Clean Code

Aspek Clean Code	Masalah Sebelum	Perbaikan	Dampak
Meaningful Names	Variabel singkat seperti \$q_id, \$val, \$res kurang ideal untuk pembelajaran	Gunakan \$questionId, \$answerValue, \$score, \$resultLabel	Maksud kode lebih cepat dipahami
Small Functions	AdminController::index() memuat banyak query dan data shaping	Pecah ke AdminDashboardService dan repository	Method controller lebih pendek
Avoid Duplication	Submit MSLQ dan AMS memiliki pola berulang	Gunakan QuestionnaireSubmissionService	Perubahan alur submit cukup di satu tempat
Clear Responsibility	Controller mencampur request, query, skoring, histori, dan output	Pisahkan ke controller, service, repository, validator, presenter	Tanggung jawab class lebih jelas
Avoid Magic Values	Tipe instrumen seperti 'VARK', 'MSLQ', 'AMS' tersebar sebagai string	Gunakan constant atau enum-like class InstrumentType	Risiko typo berkurang
Error Handling	Validasi jawaban belum terlihat eksplisit pada submit controller	Tambahkan validator yang melempar exception/domain error terkontrol	Input tidak valid lebih mudah ditangani
Query Readability	Query panjang berada langsung di controller	Pindahkan ke repository dengan nama method bermakna	Controller lebih mudah dibaca

Contoh perbaikan penamaan sederhana:

Blok kode (php)

```
// Sebelum
foreach (($_POST['answers'] ?? []) as $q_id => $val) {
    $questionModel->saveResponse($_SESSION['student_id'], 'MSLQ', $q_id, $val);
}
```

Blok kode (php)

```
// Sesudah
foreach ($answers as $questionId => $answerValue) {
    $responseRepository->saveResponse($studentId, InstrumentType::MSLQ, $questionId, $answerValue);
}
```

13. Analisis High Cohesion dan Low Coupling

Aspek	Sebelum Refactoring	Sesudah Refactoring
Cohesion Controller	Rendah sampai sedang, karena controller mengurus HTTP, query, skoring, histori, dan format output	Lebih tinggi, karena controller fokus pada request dan response
Cohesion Service	Belum tersedia lapisan service khusus	Service fokus pada business logic kuesioner, skoring, dan dashboard
Coupling Database	Controller sering memanggil <code>Database::getInstance()</code> dan query langsung	Controller tidak bergantung langsung pada SQL
Coupling Instrumen	VARK/MSLQ/AMS dikelola dengan controller/method terpisah yang mirip	Instrumen dikelola melalui service dan konfigurasi tipe instrumen
Maintainability	Perubahan aturan skoring atau laporan berisiko menyentuh banyak file	Perubahan lebih terlokalisasi pada service/repository
Testability	Controller sulit diuji tanpa database	Service dapat diuji dengan repository mock/fake

Peningkatan cohesion terjadi karena setiap class memiliki fokus yang lebih jelas. Penurunan coupling terjadi karena controller tidak lagi mengetahui detail tabel dan query. Struktur ini juga membantu mahasiswa memahami bahwa refactoring bukan hanya "memindahkan kode", tetapi memperjelas batas tanggung jawab.

14. Bukti Aplikasi Tetap Berjalan

Karena dokumen ini dibuat sebagai contoh laporan tanpa mengubah kode aplikasi riset, pengujian "sesudah refactoring" harus dilakukan pada branch latihan terpisah. Bagian ini menunjukkan format bukti yang perlu diisi mahasiswa setelah menerapkan refactoring pada salinan/branch non-produksi.

14.1 Lingkungan Uji

Komponen	Nilai
Runtime	Docker
URL Aplikasi	<code>http://localhost:8085</code>
URL Admin	<code>http://localhost:8085/admin</code>
URL API Hasil	<code>http://localhost:8085/api/results</code>
Database	MySQL 8.0
Port Database Host	3308
phpMyAdmin	<code>http://localhost:8086</code>

14.2 Perintah Verifikasi

Blok kode (powershell)

```
docker compose up -d
```

Blok kode (powershell)

```
php -l app/Controllers/VarkController.php
php -l app/Controllers/MslqController.php
php -l app/Controllers/AmsController.php
php -l app/Controllers/ResultsController.php
php -l app/Controllers/AdminController.php
php -l app/Models/Student.php
php -l app/Models/Question.php
```

Blok kode (powershell)

```
curl http://localhost:8085/
curl http://localhost:8085/api/results
```

14.3 Tabel Bukti Fungsional

No	Fitur yang Diuji	Kondisi Sebelum	Kondisi Sesudah Refactoring	Status
1	Login mahasiswa	Berjalan pada baseline aplikasi	Diisi setelah uji branch refactoring	Perlu diuji
2	Login admin	Berjalan pada baseline aplikasi	Diisi setelah uji branch refactoring	Perlu diuji
3	Pengisian VARK	Berjalan melalui VarkController	Diisi setelah uji branch refactoring	Perlu diuji
4	Pengisian MSLQ	Berjalan melalui MslqController	Diisi setelah uji branch refactoring	Perlu diuji
5	Pengisian AMS	Berjalan melalui AmsController	Diisi setelah uji branch refactoring	Perlu diuji
6	Penyimpanan respons	Menggunakan tabel responses	Diisi setelah uji branch refactoring	Perlu diuji
7	Perhitungan hasil	Menggunakan Student dan ResultsController	Diisi setelah uji branch refactoring	Perlu diuji
8	Dashboard admin	Menggunakan AdminController::index()	Diisi setelah uji branch refactoring	Perlu diuji
9	API /api/results	Mengembalikan JSON hasil kuesioner	Diisi setelah uji branch refactoring	Perlu diuji

14.4 Placeholder Screenshot

Gunakan screenshot aktual dari branch latihan setelah refactoring.

Blok kode (markdown)

```
! [Screenshot Login] (screenshots/login.png)
! [Screenshot Dashboard Mahasiswa] (screenshots/dashboard-mahasiswa.png)
! [Screenshot Form VARK] (screenshots/form-vark.png)
! [Screenshot Hasil Kuesioner] (screenshots/hasil-kuesioner.png)
! [Screenshot Dashboard Admin] (screenshots/dashboard-admin.png)
! [Screenshot API Results] (screenshots/api-results.png)
```

15. Kesimpulan

Berdasarkan analisis kode, Portal Kuesioner Psikometrik PointMarket sudah menggunakan struktur MVC sederhana yang memisahkan controller, model, view, core, dan helper. Aplikasi juga memiliki modul psikometrik nyata, yaitu VARK, MSLQ, dan AMS, serta endpoint API untuk hasil kuesioner.

Namun, beberapa bagian masih dapat diperbaiki untuk kepentingan maintainability. Controller submit instrumen, controller hasil, dan controller dashboard admin masih memuat terlalu banyak tanggung jawab. Refactoring yang disarankan adalah menambahkan lapisan service, repository, validator, dan presenter/formatter agar kode lebih modular.

Dengan pemisahan tersebut, controller menjadi lebih ringkas, logika skoring lebih mudah diuji, query laporan lebih terpusat, dan penambahan instrumen psikometrik baru dapat dilakukan dengan risiko perubahan yang lebih rendah. Refactoring juga membantu kita memahami penerapan praktis SOLID, Clean Code, High Cohesion, dan Low Coupling pada aplikasi web nyata.

16. Lampiran

16.1 Link Repository

Diisi sesuai repository kelas atau repository latihan yang digunakan mahasiswa.

Contoh:

Blok kode (text)

```
https://github.com/nama-kelas/portal-kuesioner-psikometrik
```

16.2 Branch Sebelum dan Sesudah Refactoring

Jenis Branch	Nama Branch
Branch sebelum refactoring	before-refactoring
Branch sesudah refactoring	after-refactoring

16.3 Daftar Commit Penting

No	Commit	Deskripsi Perubahan
1	[hash]	Menambahkan QuestionnaireResponseValidator
2	[hash]	Memindahkan alur submit ke QuestionnaireSubmissionService
3	[hash]	Memindahkan query hasil ke QuestionnaireResultRepository
4	[hash]	Memindahkan query dashboard ke AdminDashboardRepository
5	[hash]	Merapikan CRUD pertanyaan melalui QuestionAdminService

16.4 Daftar File yang Dianalisis

No	File	Peran
1	README.md	Deskripsi fitur dan struktur proyek
2	public/index.php	Entry point aplikasi
3	app/Core/Router.php	Routing sederhana berbasis controller/method
4	app/Core/Database.php	Koneksi PDO singleton
5	app/Controllers/VarkController.php	Controller instrumen VARK
6	app/Controllers/MslqController.php	Controller instrumen MSLQ
7	app/Controllers/AmsController.php	Controller instrumen AMS
8	app/Controllers/ResultsController.php	Tampilan hasil psikometrik mahasiswa
9	app/Controllers/AdminController.php	Dashboard dan manajemen admin
10	app/Controllers/ApiController.php	Endpoint JSON hasil kuesioner
11	app/Models/Question.php	Pengambilan pertanyaan dan penyimpanan respons
12	app/Models/Student.php	Login, sinkronisasi, dan perhitungan hasil
13	app/Models/Setting.php	Status buka/tutup instrumen
14	database/db_schema.sql	Skema tabel utama
15	dev-resources/docs/swagger.yaml	Dokumentasi API

16.5 Rekomendasi Struktur Folder Setelah Refactoring

Blok kode (text)

```
app/
|-- Controllers/
|-- Core/
|-- Helpers/
|-- Models/
```

```
|-- Repositories/  
|   |-- AdminDashboardRepository.php  
|   |-- QuestionRepository.php  
|   |-- QuestionnaireResultRepository.php  
|   |-- QuizHistoryRepository.php  
|   |-- ResponseRepository.php  
|-- Services/  
|   |-- AdminDashboardService.php  
|   |-- PsychometricResultService.php  
|   |-- PsychometricScoringService.php  
|   |-- QuestionnaireSubmissionService.php  
|   |-- QuestionAdminService.php  
|-- Validators/  
|   |-- QuestionnaireResponseValidator.php  
|   |-- QuestionRequestValidator.php  
|-- Presenters/  
|   |-- QuestionnaireResultPresenter.php  
|-- Views/
```